FIAP ON

BIG DATA

PROJETO BIG DATA

6

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 – Elementos principais de um Pipeline.....6
Figura 2 – Exemplo de Pipeline.....7
Figura 3 – Exemplo de Pipeline Big Data Simplificado.....7
Figura 4 – Fontes de dados do projeto.....9
Figura 5 – Dados são copiados para o HDFS.....10
Figura 6 – Pipeline completo da Prova de Conceito.....13

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Código Python para MAPPER.PY 11
Código-fonte 2 – Código Python para REDUCER.PY.....12

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 – Comando Hadoop para criação do diretório mensagens...9
Comando de prompt 2 – Comando Hadoop para criação do diretório mensagens...10
Comando de prompt 3 – Comando para executar as funções MAP e REDUCE 12

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

PROJETO BIG DATA.....6
1 INTRODUÇÃO 6
2 PIPELINE 6
3 O CENÁRIO 7
4 PROVA DE CONCEITO 8
REFERÊNCIAS.....14

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

PROJETO BIG DATA

1 INTRODUÇÃO

A maioria dos negócios que envolvem grandes volumes de dados buscam formas de integrar dados de várias fontes para, com isso, obter insights que possam gerar alguma vantagem competitiva.

Um pipeline de dados é um conjunto de ações que ingerem dados brutos de fontes diferentes e movem os dados para um destino para armazenamento e análise. Um pipeline também pode incluir filtragem e recursos que fornecem resiliência contra falhas.

2 PIPELINE

Para Hazelcast (2021), um pipeline de dados é uma série de etapas de processamento de dados. Se os dados não estiverem carregados na plataforma de dados, eles serão ingeridos no início do pipeline. Depois, há uma série de etapas em que cada etapa fornece uma saída que é a entrada para a próxima etapa. Isso continua até que o pipeline seja concluído. Em alguns casos, etapas independentes podem ser executadas em paralelo.

Figura 1 – Elementos principais de um Pipeline
Fonte: Elaborado pelo autor (2022)

Os pipelines de dados consistem em três elementos principais: uma origem, uma etapa ou etapas de processamento e um destino. Em alguns pipelines de dados, o destino pode ser chamado de coletor.

Os pipelines de dados permitem o fluxo de dados de um aplicativo para um data warehouse, de um data lake para um banco de dados analítico ou para um sistema de processamento de pagamentos, por exemplo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

De acordo com Teachtarget (2021), os dados gerados em um sistema ou aplicativo podem fornecer dados para vários pipelines de dados, e esses pipelines, por sua vez, podem ter vários outros pipelines ou aplicativos que dependem de suas saídas.

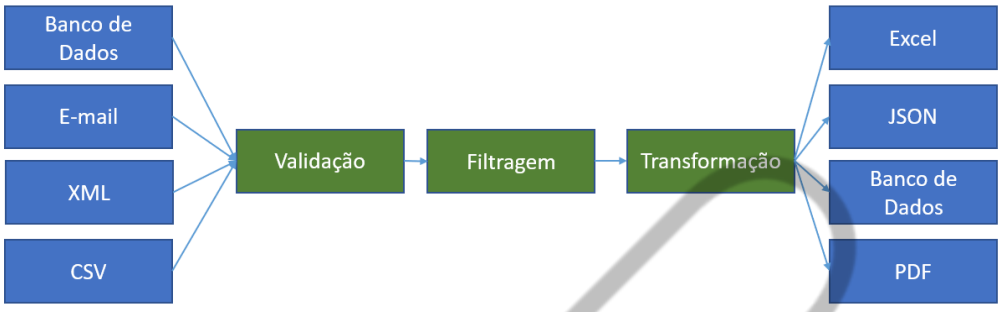

Figura 2 – Exemplo de Pipeline
Fonte: Elaborado pelo autor (2022)

Na figura 2, o pipeline mostra diversas fontes de dados sendo lidas, seus dados sendo validados, filtrados segundo algum critério e transformados para serem usados na geração de novos conjuntos de dados. No exemplo, o fluxo para nesse ponto, mas, as novas fontes de dados poderiam ser lidas por outros aplicativos e, nesse caso, indicaríamos isso no pipeline.

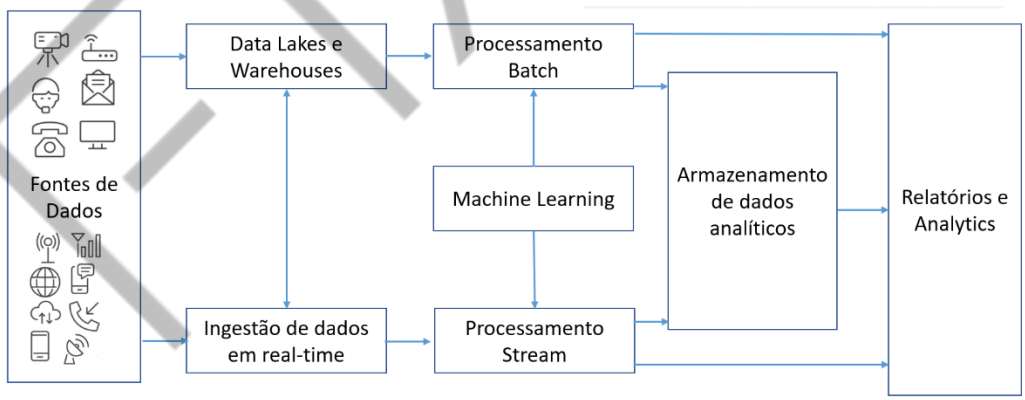

Figura 3 – Exemplo de Pipeline Big Data Simplificado
Fonte: Elaborado pelo autor (2022)

3 O CENÁRIO

Um hospital que atende um grupo de cidades pequenas no interior de São Paulo possui um processo de controle de qualidade de atendimento baseado em um formulário de pesquisa de satisfação do cliente.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Até o início da pandemia de covid-19, esse formulário era preenchido em formulário físico fornecido ao paciente no momento da alta. Além das perguntas objetivas, o formulário possuía um espaço para observações onde o paciente poderia fornecer comentários extras sobre sua estadia.

Ao final do dia, os formulários eram reunidos e transcritos para um sistema digital, as pessoas que transcreviam o formulário, além de digitarem as observações também analisavam os comentários e indicavam aqueles que fossem pertinentes para melhorar a qualidade do atendimento.

Com a chegada da pandemia de covid-19, esse formulário passou a ser fornecido de forma digital. Os dados dos formulários passaram a ser armazenados em arquivos em formato .CSV e disponibilizados para o sistema. Essa mudança agilizou o processo de carga de dados e, para otimizar seus recursos humanos, a instituição decidiu readequar as atividades dos transcritores transferindo-os para outras áreas; como consequência dessa readequação de funções, a análise dos comentários deixou de ser feita no momento da entrada dos dados.

Outra consequência da pandemia foi que o hospital começou a receber mensagens de diversas fontes. Apesar das mensagens serem importantes para o controle de qualidade, o sistema não estava preparado para processá-las e foram sendo armazenadas sem que fossem analisadas com o devido cuidado.

A instituição percebeu que os dados das observações e mensagens eram uma fonte importante de informações e seria fundamental analisar esses dados. Para uma primeira análise exploratória precisam reunir os dados e tratar as palavras mais frequentes em busca de um padrão para análise.

4 PROVA DE CONCEITO

Como demonstração da viabilidade do projeto, foi montado uma prova de conceito. Todo o processo foi feito manualmente e, em uma etapa posterior, seria automatizada.

O primeiro passo para a solução foi a identificação do formato dos arquivos que o hospital mantém armazenado. Foram identificados quatro formatos de armazenamento: CSV, Arquivos Sequenciais, Avro e Parquet.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 4 – Fontes de dados do projeto
Fonte: Elaborado pelo autor (2022)

Uma vez identificadas as múltiplas fontes de dados, foram copiadas para o ambiente HDFS. Existem várias ferramentas que podem ser usadas para a transferência dos dados e para sua automação, podemos citar as ferramentas SQOOP e FLUME como exemplo. Nesse primeiro momento, a transferência de dados foi feita manualmente.

Foi criado um diretório de nome **mensagens** no HDFS utilizando o comando:

```
hadoop fs -mkdir mensagens
```

Comando de prompt 1 – Comando Hadoop para criação do diretório mensagens
Fonte: Elaborado pelo autor (2022)

Em seguida, os arquivos foram copiados para o diretório mensagens criado no Hadoop. Todos os arquivos de mensagens iniciavam pela palavra mensagem, sendo assim, foi possível usar o padrão mensagem*.* para agilizar a cópia de todos os arquivos para o diretório mensagens.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
hadoop fs -copyFromLocal mensagem*.* ./mensagens
```

Comando de prompt 2 – Comando Hadoop para criação do diretório mensagens
Fonte: Elaborado pelo autor (2022)

Depois dessa operação, as mensagens estão disponíveis no ambiente HDFS.

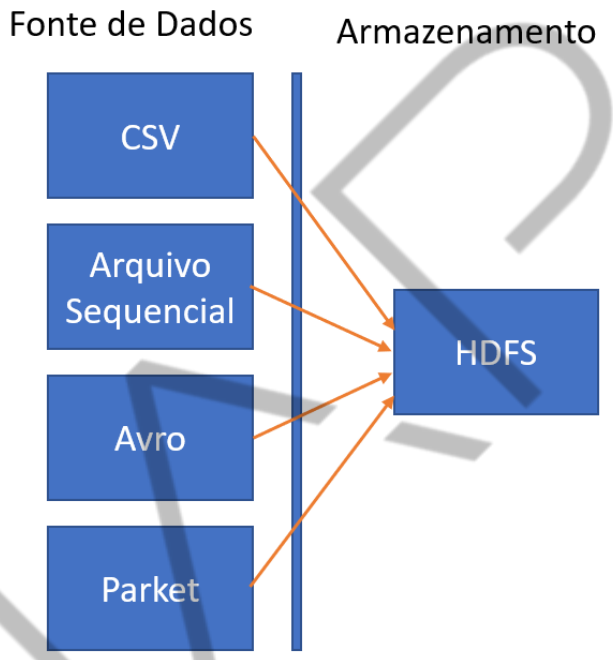

Figura 5 – Dados são copiados para o HDFS
Fonte: Elaborado pelo autor (2022)

Os arquivos podem ser processados por uma função de MapReduce para pesquisar os dados e retornar à sumarização dos dados. A maioria das linguagens de programação já possui uma biblioteca com as funções MAP e REDUCE. Aqui, para efeito didático, criaremos dois programas em Python para simular as operações.

O primeiro programa, mapper.py, lerá os dados STDIN e dividirá as linhas em palavras, produzindo, como saída, a contagem individual de cada palavra.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
#!/usr/bin/env python
"""mapper.py"""

import sys

# entrada vem de STDIN (entrada padrão)
for line in sys.stdin:
    # remove os espaços em branco iniciais e finais
    line = line.strip()
    #divide a linha em palavras
    words = line.split()
    # incrementa contadores
    for word in words:
       # Escreve os resultados em STDOUT (saída padrão);
       # a saída será a entrada para a etapa de Reducer,
       # ou seja, a entrada para reducer.py
       #
       # delimitado por tabulação;
       # a contagem de palavras é 1

        print '%s\t%s' % (word, 1)
```

Código-fonte 1 – Código Python para MAPPER.PY
Fonte: IBM (2022)

O segundo programa, reducer.py, lerá a saída de mapper.py de STDIN (entrada padrão) e agregará a ocorrência de cada palavra, gravando a saída final em STDOUT.

```
#!/usr/bin/env python
"""reducer.py"""

from operator import itemgetter
import sys

current_word = None
current_count = 0
word = None

# entrada vem de STDIN (entrada padrão)
for line in sys.stdin:
    # remove espaços em branco à direita e à esquerda
    line = line.strip()

    # analisa a entrada que recebemos de mapper.py
    word, count = line.split('\t', 1)
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        # converte count (no momento é texto) para int
        try:
            count = int(count)
        except ValueError:
            # se count não for um número
            # ignore/descarta a linha
            continue

        if current_word == word:
            current_count += count
        else:
            if current_word:
                # grava o resultado em STDOUT
                print     '%s\t%s'     %     (current_word,
current_count)
            current_count = count
            current_word = word

    if current_word == word:
        print '%s\t%s' % (current_word, current_count)
```

Código-fonte 2 – Código Python para REDUCER.PY
Fonte: IBM (2022)

Para executar nossos programas manualmente usaremos:

```
cat mensagens | python mapper.py | sort -k1 | python
reducer.py
```

Comando de prompt 3 – Comando para executar as funções MAP e REDUCE
Fonte: Elaborado pelo autor (2022)

Como saída, temos um conjunto com a contagem das palavras usadas em todas as mensagens, esse arquivo pode ser usado em nossa análise exploratória para encontrar os padrões necessários para o departamento de qualidade. Também pode passar por um filtro, limpando as palavras irrelevantes e tratar a saída para ser usada em uma ferramenta de auto service BI.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

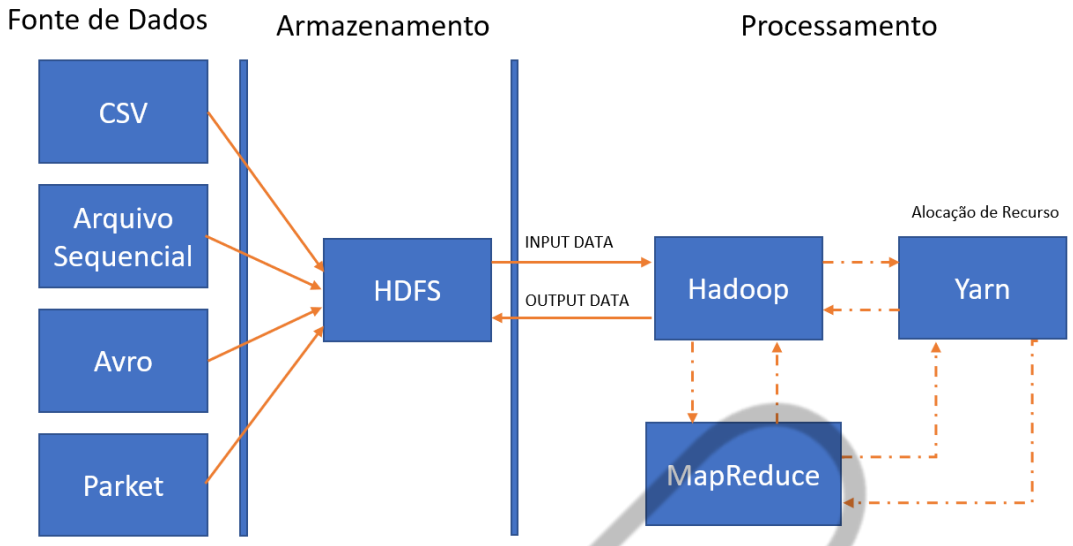

Figura 6 – Pipeline completo da Prova de Conceito
Fonte: Elaborado pelo autor (2022)

Este pequeno projeto foi usado como prova de conceito para o desenvolvimento do projeto, em etapas posteriores foram usadas ferramentas para a automatização dos processos, programas desenvolvidos em linguagem JAVA para as operações do sistema e desenvolvido um dashboard para análise dos dados usando a ferramenta PowerBI.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

What Is a Data Pipeline?. **Hazelcast**. 14 dez. 2021. Disponível em: <https://hazelcast.com/glossary/data-pipeline/>. Acesso em: 08 maio 2022.

How to build an all-purpose big data pipeline architecture. **TeachTarget**. 01 abr. 2021. Disponível em: <https://www.techtarget.com/searchdatamanagement/feature/How-to-build-an-all-purpose-big-data-pipeline-architecture>. Acesso em: 08 maio 2022.

Hadoop example: Hello World with Java, Pig, Hive, Flume, Fuse, Oozie, and Sqoop with Informix, DB2, and MySQL. **IBM**. 27 set 2012. Disponível em: https://developer.ibm.com/tutorials/dm-1209hadoopbigdata/. Acesso em: 14 jun. 2022.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com